GlitchProber: Advancing Effective Detection and Mitigation of Glitch Tokens in Large Language Models

Zhibo Zhang*
Huazhong University of Science and Technology
Wuhan, China
zhangzhibom@hust.edu.cn

Wuxia Bai*
Huazhong University of Science and Technology
Wuhan, China
wuxiabai@hust.edu.cn

Yuxi Li*
Huazhong University of Science and Technology
Wuhan, China
yuxili@hust.edu.cn

Mark Huasong Meng
Technical University of Munich
Munich, Germany
huasong.meng@gmail.com

Kailong Wang†
Huazhong University of Science and Technology
Wuhan, China
wangkl@hust.edu.cn

Ling Shi
Nanyang Technological University
Singapore, Singapore
ling.shi@ntu.edu.sg

Li Li
Beihang University
Beijing, China
lilicoding@ieee.org

Jun Wang
Beihang University
Beijing, China
junwang.lu@gmail.com

Haoyu Wang
Huazhong University of Science and Technology
Wuhan, China
haoyuwang@hust.edu.cn

arXiv:2408.04905v2 [cs.CL] 23 Sep 2024

ABSTRACT

Large language models (LLMs) have achieved unprecedented success in the field of natural language processing. However, the black-box nature of their internal mechanisms has brought many concerns about their trustworthiness and interpretability. Recent research has discovered a class of abnormal tokens in the model's vocabulary space and named them "glitch tokens". Those tokens, once included in the input, may induce the model to produce incorrect, irrelevant, or even harmful results, drastically undermining the reliability and practicality of LLMs.

In this work, we aim to enhance the understanding of glitch tokens and propose techniques for their detection and mitigation. We first reveal the characteristic features induced by glitch tokens on LLMs, which are evidenced by significant deviations in the distributions of attention patterns and dynamic information from intermediate model layers. Based on the insights, we develop GlitchProber, a tool for efficient glitch token detection and mitigation. GlitchProber utilizes small-scale sampling, principal component analysis for accelerated feature extraction, and a simple classifier for efficient vocabulary screening. Taking one step further, GlitchProber rectifies abnormal model intermediate layer values to mitigate the destructive effects of glitch tokens. Evaluated on five mainstream open-source LLMs, GlitchProber demonstrates higher efficiency, precision, and recall compared to existing approaches, with an average F1 score of 0.86 and an average repair rate of 50.06%. GlitchProber unveils a novel path to address the challenges posed by glitch tokens and inspires future research toward more robust and interpretable LLMs. Our code is available at https://github.com/LLM-Integrity-Guard/GlitchProber.

CCS CONCEPTS

• **Computing methodologies** → *Knowledge representation and reasoning*.

KEYWORDS

LLM security, Glitch token, LLM analysis

ACM Reference Format:
Zhibo Zhang, Wuxia Bai, Yuxi Li, Mark Huasong Meng, Kailong Wang, Ling Shi, Li Li, Jun Wang, and Haoyu Wang. 2024. GlitchProber: Advancing Effective Detection and Mitigation of Glitch Tokens in Large Language Models. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3691620.3695060

1 INTRODUCTION

In the field of Natural Language Processing (NLP), large language models (LLMs) like GPT-4 [2], Gemini [31, 34], and Claude 3 [4] have demonstrated near-human-level text generation capabilities. Their exceptional performance has led to widespread adoption [16, 35, 41]. When using these models, users provide a prompt, which the model's tokenizer breaks down into a series of discrete tokens. These tokens are the fundamental units of information processing for the model, playing a crucial role in the usage of LLMs. Recent research [12, 13, 26, 28, 29, 32], however, has shown that some "glitch tokens" exist in the vocabulary of LLMs. Once included in a prompt,

*Co-first author with equal contribution.
†Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '24, October 27-November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1248-7/24/10…$15.00
https://doi.org/10.1145/3691620.3695060

these special tokens can potentially lead to model errors, such as misunderstanding user intent, refusing to answer, or generating irrelevant or harmful text. Therefore, thorough analysis and detection of these glitch tokens are crucial to ensure the reliability and safety of LLMs.

To tackle the glitch tokens issue, one notable method recently is presented by Li et al. [21]. This typical and intuitive solution involves studying the characteristics of glitch tokens in the word embedding space of LLMs and accordingly developing detection techniques. They discovered that glitch tokens tend to cluster in the embedding space and proposed an iterative clustering-based technique called GlitchHunter for efficient glitch token detection.

Although there has been progress in detecting glitch tokens, there still lacks an efficient and precise detection of glitch tokens universally applicable in different LLMs. Furthermore, existing approaches primarily focus on detection, however, how to fix the issues caused by glitch tokens in the usage of LLMs remains an open question. Several limitations contribute to the aforementioned challenges:

(1) The exhaustive search method of checking vocabulary is simple and intuitive but incurs significant time costs with large token sets, making it inefficient for practical use.
(2) Existing detection methods primarily identify glitch tokens based on features like word frequency and word vectors. However, these features do not deeply explore the mechanisms by which glitch tokens impact model behaviors, resulting in poor detection accuracy and generalization performance.
(3) Current research primarily focuses on detecting glitch tokens rather than how to fix them. While detection can identify issues, it does not eliminate the negative impact of glitch tokens on model performance, limiting its practical value.

Our work. To address these existing challenges and bridge the gap, in this work, we investigate the internal structure of LLMs to explore the differences between glitch tokens and normal tokens. Specifically, through empirical study, we discovered significant differences between glitch tokens and normal tokens in terms of the attention patterns and dynamic information of multi-layer perceptron (MLP) modules within transformer-based LLMs. This discovery reveals the adversarial impact of glitch tokens on the internal mechanisms of the model, indicating that glitch tokens introduce abnormal interference and noise to neural networks. This hinders the model from correctly understanding and processing the semantic information carried by these tokens, ultimately leading to erroneous outputs.

From these findings, we gain an insight that the glitch tokens can be efficiently detected due to the deviated distributions of intermediate layers' outputs caused by them, and accordingly their impact can be effectively mitigated by proactively rectifying those abnormal outputs. Based on this insight, we propose a new method for glitch token detection and fix called GlitchProber. For glitch token detection, GlitchProber first samples a small subset of manually labeled glitch tokens as the sample set, and extracts the outputs of these tokens from the intermediate layers, specifically, the attention scores of the attention patterns and MLP status. It then applies Principal Component Analysis (PCA) [1] dimensionality reduction to the intermediate layers' outputs and obtains a feature representation matrix for the sample set. This matrix, along with the corresponding class labels, is used to train a Support Vector Machine (SVM) [9] classifier, which can subsequently be employed for glitch token detection. To fix the glitch tokens, GlitchProber analyzes the activation value range of normal tokens in the intermediate MLP status and rectifies the activation states of glitch tokens. Specifically, it aims to adjust the activation patterns of glitch tokens to be closer to those of normal tokens, and thereby minimize their impact on the model's output. Our work is published on our website [15].

Contributions. We summarize our key contributions as follows:

- **Empirical Study Exploring the Internal Impact of Glitch Tokens on LLMs.** We conduct a comprehensive and systematic empirical study on how glitch tokens and normal tokens manifest at the structural level across different LLMs. One of our key findings is that glitch tokens can trigger abnormal values in a model's attention patterns and MLP status.
- **Effective Glitch Token Detection.** Our evaluation on five representative open source LLMs demonstrates that GlitchProber can save approximately 40% of time in glitch token detection compared to the state-of-the-art approaches. Additionally, GlitchProber exhibits a significant improvement in detection accuracy.
- **Effective Glitch Token Fixing.** In terms of fix, GlitchProber successfully repairs an average of 7,758 tokens across the five LLMs. It achieves an average repair rate of 50.06%, significantly outperforming the baseline approach. Our results demonstrate the effectiveness of the proposed fix strategy by adjusting the intermediate values of glitch tokens in the intermediate layers.

Ethical Consideration. In this work, we recognize that glitch tokens can cause abnormal responses from LLMs, potentially affecting their usage. However, we strictly adhere to ethical principles and do not condone any abuse or exploitation of these findings. Our research aims to raise awareness of these risks and contribute to a more secure LLM community. We have reported our findings to the respective LLM developers and are committed to collaborating with them to develop effective defenses and mitigation strategies. By working cooperatively, we promote responsible research practices and ensure the safe and beneficial use of LLMs.

2 BACKGROUND AND RELATED WORK

2.1 Transformer-based LLMs

Self-attention [7, 25] is a core component of Transformer-based models, demonstrating strong modeling capabilities across various tasks. Given an input $X \in \mathbb{R}^{n \times d}$, where n denotes the sequence length and d denotes the dimension, self-attention linearly projects X into query, key, and value representations, i.e., Q, K, and V. The attention scores matrix A is then computed by taking the dot product between the query and key matrices, followed by a softmax normalization. The attention output is obtained by multiplying the attention scores with the value matrix.

$$A = \text{softmax}(\frac{QK^T}{\sqrt{d}}) \tag{1}$$

$$\text{Attention}(Q, K, V) = A \cdot V \tag{2}$$

To analyze the behavior of Transformer-based models during sequence processing, we introduce the concept of *attention patterns*,

which can be extracted from the corresponding row $A[n]$ of the attention scores matrix A. In autoregressive generation tasks, the attention patterns only contain the attention weights between the current token and previously generated tokens.

The MLP module in Transformer-based models employs a gating mechanism similar to that of Gated Multi-Layer Perceptrons (gMLPs) [23]. Given an input $Y \in \mathbb{R}^{n \times d}$, where n denotes the sequence length and d denotes the dimension, the MLP first projects Y to a higher-dimensional space using a linear transformation:

$$Z = YU \tag{3}$$

where $U \in \mathbb{R}^{d \times d_m}$ is a learnable weight matrix. The transformed representation Z is then split along the feature dimension into two matrices, $Z_1, Z_2 \in \mathbb{R}^{n \times d_m/2}$:

$$Z_1, Z_2 = \text{split}(Z) \tag{4}$$

An activation function σ is applied element-wise to Z_1 to obtain the *MLP gate* $\sigma(Z_1)$. The MLP gate is then multiplied element-wise with the *MLP data* Z_2 to produce the gated output $\tilde{Z}$:

$$\tilde{Z} = \sigma(Z_1) \odot Z_2 \tag{5}$$

Finally, another linear transformation is applied to map $\tilde{Z}$ back to the original dimension.

$$\text{Output} = \tilde{Z}W \tag{6}$$

where $W \in \mathbb{R}^{d_m/2 \times d}$ is another learnable weight matrix.

The MLP gate $\sigma(Z_1)$ and MLP data Z_2 in the MLP can be seen as a special variant of the spatial gating unit in Transformer-based models. These two components work together to control the information flow and capture dependencies between tokens. By extracting and analyzing the MLP gate and MLP data, we can gain insights into how the model processes and responds to different types of input within the module.

2.2 Glitch Token Phenomenon

Tokenizer plays a key role in an LLM as it transforms a continuous text sequence into a list of discrete values called tokens [39]. The tokens transformed from the training corpus form the vocabulary dictionary of LLMs, and the vocabulary dictionary in turn determines the capacity of LLMs to produce diverse and comprehensive output. The rapid advancement of LLMs has brought attention to various anomalous phenomena [10, 11, 19, 20, 22, 24, 40], one of which is the existence of "glitch tokens". These tokens exhibit anomalies in constructing the expected semantics, and are subsequently reflected in the abnormal and unexpected decoding in the LLM's output.

The glitch token phenomenon, first explored on the Lesswrong website, refers to anomalous tokens such as "SolidGoldMagikarp" and "petertodd" that cause unexpected and inaccurate results in language models like GPT-2 and GPT-J [26, 28, 29, 32]. Subsequent research examined the characteristics and instability of these tokens, revealing that even subtle changes in prompts can lead to significant differences and hallucinations in the generated results. The discovery of "polysemous" tokens, which produce different responses to repeated requests, further highlighted the prevalence and variability of the glitch token phenomenon in LLMs [27].

Recently, Li et al. [21] systematically investigated the glitch tokens with a proposed taxonomy covering their types and symptoms. They proposed three tasks, namely repetition, length and spelling, in their study to recognize glitch tokens. Their observation of the clustering distribution of glitch tokens in the word embedding spaces offers a novel perspective on the automatic identification of glitch tokens, making systematic detection feasible in LLMs containing billions, or even tens of billions of parameters.

The glitch token phenomenon uncovers the limitations and instability of LLMs when processing specific tokens. In this work, we aim to conduct a systematic and in-depth investigation of this phenomenon to gain a deeper understanding of the internal mechanisms of these models. This will provide valuable insights that can contribute to enhancing the robustness and reliability of LLMs in future applications.

3 EMPIRICAL STUDY

Our empirical study aims to explore an intuitive method for detecting glitch tokens. To this end, we investigate the differences in the model's behaviors when processing glitch tokens versus normal tokens. Two research questions are raised to guide the study:

- **RQ1 (Characteristics): What differences are exhibited between glitch tokens and normal tokens at the structural level of an LLM?**
- **RQ2 (Ubiquity): Are the differences discovered in RQ1 prevalent in most LLMs?**

To address the two RQs, we investigate the internal mechanisms of LLMs by analyzing the status of each layer in the transformer forward process of prediction. This involves examining the data flow of the intermediate layers as the model processes inputs.

3.1 Experiment Setup

To better understand the impact of glitch tokens on the model's internal output generation process, we conduct a series of experiments on the Llama-2-7b-chat model [36], shortly written as Llama2. Llama2 is a language model based on the Llama architecture. In our experiments, we set the temperature to 0 to eliminate randomness and ensure consistency in the model's responses. All other configurations are default.

We employ a unified approach to determine whether each token is glitchy or normal in this study based on existed definition of glitch tokens [12]. While the symptoms of glitch tokens may vary across different tasks, we consistently utilize a repetition task to construct input sequences for glitch token identification.

In the context of our work, a **repetitive task** refers to a specifically designed experimental procedure to test the fidelity of a language model in reproducing input tokens. This task is utilized primarily for the identification and categorization of tokens based on their performance when repetitively prompted. Specifically, the repetitive task involves the following steps:

(1) Formulating a prompt that requires the model to duplicate a specific token. The typical prompt structure is, "Can you repeat the token '$\{token\}$' and return it back to me?" This format is deliberately chosen to minimize contextual influence and focus purely on the token reproduction capability of the model.